

Node.js Worker Threads - Interview Notes

Overview

Worker Threads in Node.js allow you to run JavaScript operations in parallel threads, enabling true multi-threading for CPU-intensive tasks. This is crucial for preventing blocking operations from affecting the main event loop.

Why Worker Threads?

The Problem: Node.js Single-Threaded Nature

- **Node.js runs on a single thread** (the main thread/event loop)
- **CPU-intensive operations block the event loop**, preventing other requests from being processed
- Example: A heavy computation blocks all incoming requests until it completes

The Solution: Worker Threads

- Offload CPU-intensive tasks to separate threads
 - Keep the main thread free to handle incoming requests
 - Achieve true parallelism for computational tasks
-

Key Concepts

1. Main Thread vs Worker Thread

Main Thread:

- Handles the event loop
- Processes incoming HTTP requests
- Should remain non-blocking
- Creates and manages worker threads

Worker Thread:

- Separate JavaScript execution environment
- Runs independently from the main thread
- Has its own V8 instance and event loop
- Perfect for CPU-intensive operations

2. libuv

- **libuv** is Node.js's underlying C library that handles asynchronous I/O operations
 - Manages thread pool for async operations (file system, DNS, crypto)
 - Worker threads are **different** from libuv's thread pool
 - Worker threads give you explicit control over threading
-

Code Breakdown

Worker Thread File (`worker.js`)

```
import { parentPort } from "node:worker_threads";  
// parentPort: Communication channel to send messages back to main thread  
  
let count = 0;  
  
// CPU-intensive operation (blocking by nature)  
for (let i = 0; i < 10000000000; i++) {  
  count++;  
}  
  
// Send result back to main thread  
parentPort.postMessage(count);
```

Key Points:

- `parentPort` is the bridge between worker and main thread
 - This operation would normally block the entire Node.js process
 - When run in a worker, it only blocks that worker thread
 - `postMessage()` sends data back to the main thread (similar to `emit()`)
-

Main Application File (`app.js`)

```
import express from "express";
import { Worker } from "worker_threads";

const app = express();

app.get("/blocking", (req, res) => {
  let startTime = Date.now();

  // Create a new worker thread
  let worker = new Worker("./worker.js");

  // Listen for messages from worker thread
  worker.on("message", (data) => {
    res.status(200).json({
      success: true,
      count: data,
      message: "Blocking",
      timeTaken: (Date.now() - startTime) / 1000,
    });
  });

  // Handle worker errors
  worker.on("error", (err) => {
    console.log(err);
  });
});

app.get("/non-blocking", (req, res) => {
  let startTime = Date.now();
  res.status(200).json({
    success: true,
    message: "Non blocking",
    timeTaken: Date.now() - startTime,
  });
});

app.listen(9000, (err) => {
  if (err) console.log(err);
  console.log("Server running");
});
```

How It Works: Step-by-Step

Scenario 1: Without Worker Threads

1. Request comes to `/blocking` endpoint
2. Heavy computation starts on main thread
3. **Event loop is blocked** - no other requests can be processed
4. After 10+ seconds, computation completes
5. Response is sent
6. Other requests can now be processed

Result: All requests wait in queue until blocking operation completes

Scenario 2: With Worker Threads

1. Request comes to `/blocking` endpoint
2. New worker thread is created
3. Heavy computation runs in **separate thread**
4. **Main thread remains free** - continues processing other requests
5. When worker completes, it sends message via `parentPort.postMessage()`
6. Main thread receives message via `worker.on("message")`
7. Response is sent to client

Result: Multiple requests can be processed simultaneously

Testing the Difference

Test Steps:

1. Start the server: `node app.js`
2. Open two browser tabs
3. **Tab 1:** Hit `http://localhost:9000/blocking` (takes ~10 seconds)
4. **Tab 2:** Immediately hit `http://localhost:9000/non-blocking`

Expected Results:

With Worker Threads:

- Tab 2 responds immediately (~1ms)
- Tab 1 responds after ~10 seconds
- Both endpoints work independently

Without Worker Threads (blocking on main thread):

- Tab 2 would wait for Tab 1 to complete
 - Both would take ~10 seconds
-

Communication Between Threads

Main Thread → Worker Thread

```
const worker = new Worker("./worker.js", {  
  workerData: { initialValue: 100 } // Pass data to worker  
});
```

Worker Thread → Main Thread

```
// In worker.js
parentPort.postMessage({ result: count, status: "completed" });
```

Main Thread Receiving Data

```
worker.on("message", (data) => {
  console.log(data); // { result: 10000000000, status: "completed" }
});
```

Important Concepts for Interviews

1. When to Use Worker Threads?

Use Cases:

- Heavy computational tasks (image processing, video encoding)
- Complex calculations (cryptography, data processing)
- CPU-bound operations that would block the event loop
- Operations that take more than a few milliseconds

Don't Use For:

- I/O operations (file reading, database queries) - use async/await
- Network requests - libuv handles these efficiently
- Simple operations that complete quickly

2. Worker Thread vs Cluster Module

Feature	Worker Threads	Cluster Module
Purpose	CPU-intensive tasks	Load balancing
Scope	Single process, multiple threads	Multiple processes
Memory	Shared memory possible	Separate memory
Use Case	Parallel computation	Scale across CPU cores
Communication	postMessage (fast)	IPC (slower)

3. Worker Thread vs Child Process

Feature	Worker Threads	Child Process
Overhead	Lightweight	Heavy
Startup Time	Fast	Slow
Memory	Shared ArrayBuffer	Separate
Use Case	JavaScript code	Any executable
Communication	Fast (same process)	Slower (IPC)

4. Worker Thread Lifecycle

```
// Create worker
const worker = new Worker("./worker.js");

// Worker events
worker.on("message", (data) => { /* ... */ });
worker.on("error", (err) => { /* ... */ });
worker.on("exit", (code) => { /* ... */ });

// Terminate worker
worker.terminate();
```

Advanced Concepts

1. Worker Pool Pattern

Instead of creating a new worker for each request, maintain a pool:

```
class WorkerPool {
  constructor(size, workerScript) {
    this.size = size;
    this.workers = [];
    this.queue = [];

    for (let i = 0; i < size; i++) {
      this.workers.push(new Worker(workerScript));
    }
  }

  execute(data) {
    return new Promise((resolve, reject) => {
      const worker = this.getAvailableWorker();
      worker.once("message", resolve);
      worker.once("error", reject);
      worker.postMessage(data);
    });
  }
}
```

Benefits:

- Reuse workers instead of creating new ones
 - Limit number of concurrent workers
 - Better resource management
-

2. Shared Memory with SharedArrayBuffer

```
// Main thread
const sharedBuffer = new SharedArrayBuffer(1024);
const worker = new Worker("./worker.js", {
  workerData: { sharedBuffer }
});

// Worker can directly access and modify the same memory
```

3. Two-Way Communication

```
// Main thread
worker.postMessage({ command: "start", data: someData });

worker.on("message", (msg) => {
  if (msg.status === "progress") {
    console.log(`Progress: ${msg.percent}%`);
  }
});

// Worker thread
parentPort.on("message", (msg) => {
  if (msg.command === "start") {
    // Do work and send progress updates
    parentPort.postMessage({ status: "progress", percent: 50 });
  }
});
```

Common Interview Questions

Q1: What is the difference between Worker Threads and async/await?

Answer:

- **async/await** handles I/O operations (non-blocking but still single-threaded)
 - **Worker Threads** handle CPU-intensive tasks (true multi-threading)
 - Async operations are handled by libuv's thread pool automatically
 - Worker threads require explicit creation and management
-

Q2: Can worker threads access the same variables?

Answer:

- No, each worker has its own isolated V8 instance
 - They cannot directly access variables from the main thread
 - Must use `postMessage()` to communicate
 - Can use `SharedArrayBuffer` for shared memory (advanced)
-

Q3: How many worker threads should I create?

Answer:

- Depends on CPU cores available (`os.cpus().length`)
 - Too many workers = context switching overhead
 - Best practice: Create a worker pool with size = CPU cores
 - For the example code: Creates new worker per request (not ideal for production)
-

Q4: What happens if a worker crashes?

Answer:

```
worker.on("error", (err) => {
  console.error("Worker error:", err);
  // Create new worker or handle gracefully
});

worker.on("exit", (code) => {
  if (code !== 0) {
    console.error(`Worker stopped with exit code ${code}`);
  }
});
```

Q5: Are worker threads the same as Web Workers in browsers?

Answer:

- Similar concept but different implementations
 - Node.js Worker Threads use `worker_threads` module
 - Browser Web Workers use `new Worker()` with different API
 - Both achieve parallel JavaScript execution
-

Performance Considerations

Memory Overhead

- Each worker has its own V8 instance (~30MB overhead)
- Don't create unlimited workers
- Use worker pools for frequent tasks

When NOT to Use Worker Threads

```
// BAD - Simple calculation
const worker = new Worker("./add.js"); // Overkill!

// GOOD - Use directly
const result = a + b;
```

Monitoring Workers

```
const { isMainThread, threadId } = require("worker_threads");

console.log(`Thread ID: ${threadId}`);
console.log(`Is main thread: ${isMainThread}`);
```

Real-World Use Cases

1. **Image Processing:** Resize, compress, or transform images
2. **Video Encoding:** Process video files in parallel
3. **Data Analysis:** Process large datasets
4. **Cryptography:** Hash generation, encryption
5. **Machine Learning:** Model training or inference
6. **Report Generation:** Complex PDF or Excel generation

Best Practices

1. **Use Worker Pools** - Don't create workers for every request
 2. **Handle Errors** - Always listen to `error` events
 3. **Cleanup** - Terminate workers when done
 4. **Limit Worker Count** - Match to CPU cores
 5. **Measure Performance** - Use worker threads only when beneficial
 6. **Pass Data Efficiently** - Avoid large objects in `postMessage()`
-

Summary

- **Worker Threads enable true parallelism** in Node.js
 - **Main thread stays responsive** while workers handle heavy tasks
 - **Communication via message passing** (`postMessage` , `on("message")`)
 - **Use for CPU-intensive tasks**, not I/O operations
 - **Production apps should use worker pools**, not create workers per request
 - **Critical for scalability** when dealing with computational workloads
-

Quick Reference

```
// Import
import { Worker, isMainThread, parentPort } from "worker_threads";

// Create worker
const worker = new Worker("./worker.js");

// Send to worker
worker.postMessage(data);

// Receive from worker
worker.on("message", (data) => { /* ... */ });

// In worker: Send to main
parentPort.postMessage(data);

// In worker: Receive from main
parentPort.on("message", (data) => { /* ... */ });
```

Interview Tip

When explaining worker threads in interviews:

1. Start with the **problem** (blocking event loop)
2. Explain the **solution** (parallel execution)
3. Show you understand **when to use** them
4. Mention **production considerations** (worker pools)
5. Demonstrate knowledge of **alternatives** (cluster, child_process)